

Serverautomatisering I

Dag 1: Command-line shell (kommandoskal)

Målpind for øvelsen

1. *Lærlingen kan anvende PowerShell til automatisering og fjernadministration af servere og klienter.*
2. *Lærlingen kan implementere sikkerheden korrekt i forbindelse med scripting i Powershell.*
3. **Lærlingen kan anvende de grundlæggende Cmdlets og forstår at bruge de indbyggede hjælpefunktioner i Powershell.**
4. **Lærlingen kan anvende pipelinen i Powershell.**
5. *Lærlingen kan anvende grundlæggende systemkald til WBEM (Web-Based Enterprise Management) funktioner.*
6. *Lærlingen kan anvende -whatif, -confirm og -transcript kommandoerne i Powershell.*
7. **Lærlingen kan anvende Aliases i Powershell.**
8. **Lærlingen kan oprette og bruge variabler i Powershell.**
9. *Lærlingen kan anvende datahåndtering op imod en database struktur.*

Indhold

- **Introduktion til Command-line Shell**
 - Hvad er Command-line Shell så som **PowerShell**, **CMD** og **Bash**?
 - Hvad er PowerShell ISE (Integrated Scripting Environment), og hvorfor bør du bruge VS Code med PowerShell extension.
- **Cmdlets, Functions og Pipeline**
 - Hvad er Cmdlet og Functions
 - Objekthåndtering
 - **Øvelse 1**
 - 1.1. Grundlæggende programmering med PowerShell script.
 - 1.2. Source-control din PowerShell script.
- **Objekthåndtering**
 - Cmdlet binding.
 - **Øvelse 2:** Avanceret programmering med PowerShell script
 - Med brug af Cmdlets, functions, pipeline og **objekthåndtering**.
- **Aliases og variabler**
 - Input/output med variabler og Write-Output.
 - Intro til Aliases, og opret custom aliases.
 - **Øvelse 3 (Brug af avanceret variabler: arrays og hash-tables):**
 - Programmering med PowerShell script mod indbygget Cmdlets og funktioner.
 - Programmering med tilsvarende funktionaliteter med Linux bash.

Hvad er Command-line shell (kommandoskal)

så som **PowerShell**, **CMD** og **Bash**

PowerShell, **CMD** (på Windows) og **Bash** (på Unix maskiner) er eksempler på **Command-line shell**.

- Disse er programmer, som lader brugeren interagere med operativsystemet via tekstkommandoer (scripts) i stedet for grafiske menuer. **Den kan derved så bruges til at skrive scripts til diverse automatisering af opgaver på operativsystemet.**
- **PowerShell** er udviklet af Microsoft for at give administratorer og udviklere et **mere kraftfuldt, objektorienteret værktøj** end den klassiske **CMD (Command Prompt)**.
 - PowerShell kan bruges til at:
 - **Automatisere opgaver** (f.eks. brugerstyring, serverkonfiguration)
 - **Administrere** Windows, Azure, Microsoft 365 m.m.
 - **Skrive avancerede scripts** (PowerShell-scripts har filendelsen **.ps1**)
 - **Interagere** med API'er, filer og systemtjenester på en struktureret måde
- Hvordan adskiller PowerShell sig fra CMD og Bash.
 - CMD → Simpel, tekstbaseret, gammel (DOS-arv)
 - PowerShell → Objektbaseret, moderne, cross-platform og ekstremt fleksibel
 - Bash → Funger som PowerShell, men i Linux-verdenen

Hvad er PowerShell ISE (Integrated Scripting Environment), og hvorfor bør du bruge VS Code med PowerShell extension

- Hvis du arbejder med **moderne og cross-platform scripts**, så bør du **bruge VS Code med PowerShell Extension**.
- Hvis du kun arbejder på ældre Windows-systemer og små scripts, kan **ISE** stadig være fint.

PowerShell ISE (Integrated Scripting Environment)

- Er Microsofts *klassiske* integrerede udviklingsmiljø til PowerShell og er specielt bygget til at:
 - Skrive, teste og køre PowerShell-scripts.
 - Udføre kommandoer i en interaktiv konsol.
 - Give farvekodning, IntelliSense (autofuldførelse) og debugging.
- **Funktioner**
 - Farvekodet syntax
 - Intellisense for cmdlets, parametre og variable
 - Mulighed for at køre kode-linjer direkte (F8)
 - Integreret konsolvindue
 - Debugging (breakpoints, step-through)
- **Begrænsninger**
 - Udviklingen af PowerShell ISE **er stoppet** — Microsoft vedligeholder det, men opdaterer det ikke længere.
 - Det fungerer **kun på Windows** og **kun med Windows PowerShell (v5.1 og tidligere)**.
 - Det understøtter **ikke PowerShell Core (v6+)** – altså ikke den moderne, cross-platform version.

Kort sagt:

ISE er let at bruge og fint til klassiske Windows PowerShell scripts, men det **er forældet til moderne PowerShell-arbejde**.

Visual Studio Code (VS Code) + PowerShell Extension

- VS Code er Microsofts moderne, gratis, open source editor. Den fungerer på **Windows, macOS og Linux** og kan udvides med **extensions** – herunder **PowerShell Extension**.
- **Funktioner (med PowerShell Extension)**
 - Understøtter både **Windows PowerShell** og **PowerShell Core (7+)**
 - IntelliSense, autoudfyldning og parameterhjælp
 - Syntax highlighting og fejlanalyse
 - Debugger med breakpoints
 - Integreret PowerShell terminal
 - Git-integration (til versionsstyring)
 - Fungerer sammen med moduler som Pester (til tests) og PSReadLine
- **Fordele**
 - Cross-platform (Windows/Linux/macOS)
 - Meget hurtigere udviklingstempo end ISE
 - Stærk integration med **Azure, GitHub, Docker** m.m.
 - Understøtter moderne PowerShell-funktioner og sprogudvidelser
- **Ulemper**
 - Lidt stejlere læringskurve end ISE
 - Kræver installation af PowerShell Extension manuelt

Hvad er Cmdlet, Functions (custom Cmdlet) og pipeline

Cmdlets er små, **indbygget** kommandoer i PowerShell — udviklet i .NET og indlæst som en del af PowerShell selv eller via moduler.

- De følger altid syntaksen, med **bindestreg** mellem

Verb og **Substantiv**:

Verb-Substantiv



```
Get-Service      # Henter systemtjenester
Start-Service    # Starter en tjeneste
Stop-Process     # Stopper en proces
Get-ChildItem    # Lister filer og mapper (alias: ls, dir)
```

- De udfører en enkelt, specifik opgave.
- De returnerer **objekter**, ikke bare tekst som i klassisk CMD/bash. Ex:

```
$svc = Get-Service -Name "wuauserv"
$svc | Get-Member
```

`$svc` er her en objekt, man kan “access” objekter og tilgår deres variabler/metoder som i C#.

Cmdlets bruger **pipeline** arkitektur med følgende syntaks: “|”, som definerer, at output fra venstre kommando sendes som input til højre kommando, derfor kan koden foroven også skrives som følgende: `Get-Service -Name "wuauserv" | Get-Member`

- De har indbygget hjælp, fx: `Get-Help Get-Service -Full`

Du kan bygge eget custom Cmdlet med Functions.

Functions er brugerdefinerede eller scriptdefinerede kommandoer, skrevet direkte i PowerShell-sproget.

- De bruges til at udvide PowerShell med din egen logik.
- De kan efterligne Cmdlets i opbygning (samme verb-substantiv navngivning).
- De kan genbruges i scripts, profiler eller moduler.

Dette gøre, at man kan lave sin egen “custom” Cmdlet, ved brug af “build-in” Cmdlet. Eks:

```
function Vis-AlleServicer {
    # Henter alle services på computeren og filtrerer kun de, der kører (Running)
    Get-Service | Where-Object {$_.Status -eq 'Running'}
}
```

(Kan skrives direkte i PowerShell eller i en PowerShell script og tilgås ved at eksekver scriptet)

Din custom Cmdlet kan også tilføjes til hjælp:

```
function Vis-TungeProcessor {
    <#
    .SYNOPSIS
        Viser de 5 processer med højest CPU-forbrug.
    .DESCRIPTION
        Denne funktion viser top 5 tungeste processer.
    .EXAMPLE
        Vis-TungeProcessor
    #>
        Get-Process | Sort-Object CPU -Descending | Select-Object -First 5
}
```

Øvelse 1.1 - Grundlæggende programmering (i PowerShell scripting)

Du skal prøve at lave eget custom Cmdlet med functions og vis at du kan eksekver dem ved brug af pipeline modellen.

1. Med Visual Studio Code, opret en PowerShell script: “UserManagement” til bruger registrering.

- Scripten skal indeholde en global variable “**Users**” af type list obevaring af bruger info.
- Scripten skal indeholde CRUD funktioner (metoder):
 - [Add-User](#)
 - Skal kunne kaldes med parameter: name og age som så tilføjes til listen.
 - [Get-Users](#)
 - Skal kunne kaldes med parameter: name og returner alder.
 - [Update-Users](#)
 - Skal kunne kaldes med parameter: name og age som så opdater brugerens alder i listen.
 - [Remove-User](#)
 - Skal kunne kaldes med parameter: name som så fjerner brugeren fra listen.

Hvordan du bliver bedømt:

- Du skal demonstrer brug af det oprettet script ved at fremvis hvordan du opret, updater, slet samt udskrive listen af alle burger fra din PowerShell terminal.
- Du skal demonstrer ved fremvisning hvordan du kan debug med breakpoints i din kode, f.ex. Set break point i din Add-User når du køre din script.
- Du skal demonstrer at du kan source control din projektmappe med Git og vis at du kan brug VS Code’s Git-panelet til at se ændringer samt commit og push.

Øvelse 1.2 - Source control din PowerShell project med Git

- Installer Git
 - Gå til Git for Windows: <https://git-scm.com/install/windows>
 - Download og installer programmet.
 - Under installationen skal du vælge default på nært:
 - **Use Visual Studio Code as Git's default editor**
 - **"Git from the command line and also from 3rd-party software"** → sikrer, at Git tilføjes til PATH.
 - Genstart Visual Studio Code.
- Source control din projekt mappen således:
`git init`
`git add .`
`git commit -m "Første version af UserManagement script"`

Hvad gøre [CmdletBinding()] når en PowerShell funktion implementer den?

Til øvelsen (på næste slide), skal du tilføje [CmdletBinding()] i din PowerShell function. Dette vil aktivere “common parameters” for din funktion så din funktion understøtter:

- Verbose
- Debug
- ErrorAction
- WarningAction
- InformationAction
- OutVariable
- OutBuffer

Øvelse 2: Avanceret programmering (i PowerShell scripting)

Lav en ny branch i GitHub og byg vider på koden fra øvelse 1

Add-User

- Skal være en avanceret funktion med [CmdletBinding()] så den understøtter "common parameter" som indbygget Cmdlet funktioner.
- Parametre:
 - Name (mandatory, ikke tom)
 - Age (mandatory, skal være mellem 1 og 120)
- Skal give fejl hvis brugeren allerede findes
- Skal supportere:
`New-User -Name "Peter" -Age 25`
- Skal outputte det oprettede object

Get-User

- Skal være en avanceret funktion med [CmdletBinding()] så den understøtter "common parameter" som indbygget Cmdlet funktioner.
- Parameter Name skal kunne tage **flere navne** (`string[]`)
- Skal kunne bruges i pipeline:
`"Peter","Pia" | Get-User`
- Skal støtte wildcard:
`Get-User -Name "P*"`

Update-User

- Skal være en avanceret funktion med [CmdletBinding()] så den understøtter "common parameter" som indbygget Cmdlet funktioner.
- Input kan komme fra pipeline som objekter:
`Get-User Peter | Set-User -Age 30`
- Eller via navn:
`Set-User -Name "Peter" -Age 30`
- Brug Write-Verbose til at vise hvad der bliver ændret
- Hvis brugeren ikke findes → `Write-Error`

Remove-User

- Understøt Confirm/WhatIf automatisk (via `SupportsShouldProcess`)
- Skal kunne slette via pipeline:
`Get-User Peter | Remove-User`
- Skal kunne slette flere på én gang:
`Remove-User -Name "Peter","Pia"`

Ny metode: Export-Users (Eksporter alle brugere til JSON)

- Parameter: Path
- Opretter fil hvis den ikke findes
- Output-format: JSON (brug `ConvertTo-Json`)
- Skal kunne bruges som:
`Export-Users -Path .\users.json`

Ny metode: Import-Users (Importér brugere fra JSON)

- Skal merge med eksisterende brugere uden at overskrive
- Brug Write-Verbose til at vise hvilke brugere der importeres
- Skal bruges som:
`Import-Users -Path .\users.json -Verbose`

Input/output med variabler

Input i PowerShell implementeres gennem variabler, som er et navn, du bruger til at **gemme data midlertidigt i hukommelsen**, fx tekst, tal, objekter eller hele kommandoresultater.

Output implementeres med kommando: [Write-Output](#).

- I PowerShell starter alle variabelnavne med et **dollar-tegn \$**. Eks:

```
$navn = "Anders"
```

```
$alder = 29
```

```
Write-Output "Hej, mit navn er $navn og jeg er $alder år gammel."
```

- Variabler kan gemme objekter. PowerShell-variabler kan indeholde **alt** — endda hele objekter. Eks:

```
$service = Get-Service -Name Spooler
```

```
$service.Status
```

```
$service.ServiceType
```

- Du kan altså gemme et Cmdlet-output i en variabel og arbejde med dets egenskaber direkte.
- Godt at kende, når man arbejder ved variabler i sin terminal session:
 - Se alle variabler i den aktuelle session: [Get-Variable](#)
 - Slette en variable:

```
Remove-Variable -Name navn
```

```
eller
```

```
Remove-Variable navn
```

Hvad er aliases

Alias er et kort navn eller en genvej til en længere PowerShell-kommando eller Cmdlet. Det gør det hurtigere at skrive, især ved interaktiv brug. Eks:

Alias	Cmdlet	Forklaring
ls	Get-ChildItem	Lister filer og mapper
dir	Get-ChildItem	Samme som ovenfor (Windows-style)
cd	Set-Location	Skift mappe
cp	Copy-Item	Kopier filer
mv	Move-Item	Flyt filer
rm	Remove-Item	Slet filer
gc	Get-Content	Læs filindhold
echo	Write-Output	Skriv tekst til konsol

Custom aliases kan oprettes med variabler:

[Set-Alias topCPU Get-Process](#)

[\\$top5 = topCPU | Sort-Object CPU -Descending |
Select-Object -First 5](#)

- topCPU er aliaset for Get-Process.
- \$top5 gemmer de 5 processer med højest CPU-forbrug.
- For at se de gemte 5 processer:

[\\$top5 | Format-Table -AutoSize](#)

Øvelse 3

Del 1 – PowerShell (Windows)

Udvikl et PowerShell-script, der kan:

1. Hente alle **kørende tjenester** på systemet.
2. Koble hver tjeneste til den bagvedliggende **proces** via WMI (Windows Management Instrumentation).
3. Indsamle og vise information om:
 - Servicenavn
 - CPU-forbrug
 - Hukommelsesforbrug (MB)
 - Proces-ID
4. Sortere resultaterne, så de mest ressourcekrævende tjenester vises øverst.
5. Gemme resultatet i en logfil med **tidsstempel** i filnavnet.
6. (Bonus) Tillad et parameterinput, fx -Top 5, så brugeren selv kan vælge hvor mange tjenester der skal vises.
7. **Krav:**
8. Scriptet skal være letlæseligt, med relevante kommentarer.
9. Der skal anvendes cmdlets som Get-Service, Get-Process, Get-WmiObject, Sort-Object og Select-Object.
10. Resultatet skal vises i en pæn tabel i konsollen.

Del 2 – Bash (Linux)

Udvikl et tilsvarende Bash-script, der kan:

1. Hente alle aktive (running) systemd-services med systemctl.
2. Koble hver service til dens proces-ID (MainPID).
3. For hver proces hente CPU- og hukommelsesforbrug via ps.
4. Vise resultaterne sorteret efter belastning.
5. Gemme resultatet i en logfil i brugerens hjemmemappe.
6. **Krav:**
7. Brug systemctl, awk, ps og almindelige Bash-kommandoer.
8. Tilføj kommentarer, der forklarer hvert trin i scriptet.